Lecture 12

The goal of this lecture is to understand how one can go about correcting errors in quantum states. It is not obvious at all that this is even possible, so we will first see how this works for deterministic and probabilistic bits.

Classical repetition code

The simplest way to protect information from errors is by duplicating it. This introduces a lot of redundancy, so even if some copies are perturbed or even destroyed, one can still recover the information from the remaining copies. For example, let's make 3 copies:

$$\text{Enc:} \quad \begin{matrix} 0 \to 000 \\ 1 \to 111 \end{matrix}$$

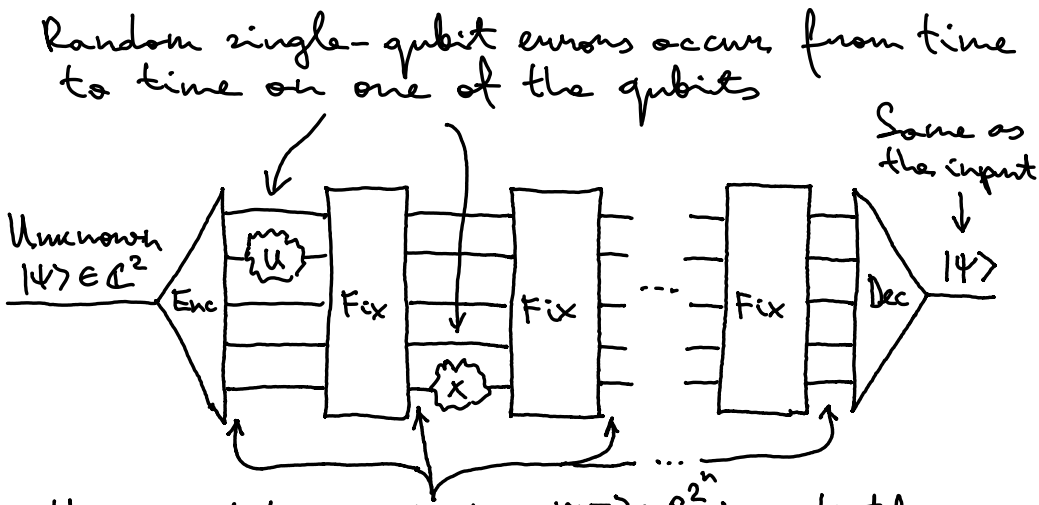
This code works well if at most one bit-flip error occurs. Alternatively, you can think of each bit getting flipped with a very small probability. E.g., 001 was most likely produced by having an error on the last bit of 000, rather than two errors – one on each of the first two bits. In general, we would decode by using the majority vote:

$$\text{Dec:} \quad \left.\begin{matrix} 000 \\ 001 \\ 010 \\ 100 \end{matrix}\right\} \to 0 \qquad \left.\begin{matrix} 111 \\ 110 \\ 101 \\ 011 \end{matrix}\right\} \to 1$$

This takes care of any single-bit error.

Problem setup

Let us state a bit more formally the problem we are trying to address. Imagine somebody gives you an unknown qubit state $|\psi\rangle \in \mathbb{C}^2$ and asks you to store it for some time or transmit it to somebody else. If the storage device or the transmission channel are not perfect, any individual qubit might get perturbed with very small probability by an arbitrary single-qubit operation $U \in U(2)$. Hence, we want to spread the information out into several qubits to protect it:



Uncorrupted encoded state $|\Psi\rangle \in \mathbb{C}^{2^n}$ immediately after encoding and each fixing operation.

We need to apply a fixing operation Fix often enough to make sure that the errors do not accumulate and we have at most one error to fix. We assume that at each time step any of the n qubits is affected by an unknown single-qubit gate with very small probability.

Obstacles to quantum error correction

At first it might seem that quantum error correction is impossible for several reasons:

- cannot duplicate information since quantum states cannot be copied,
- cannot check if an error has occurred without measuring the state, but a measurement would destroy the state,
- the errors are continuous — there are infinitely many possible errors, so we cannot hope to perfectly correct them.

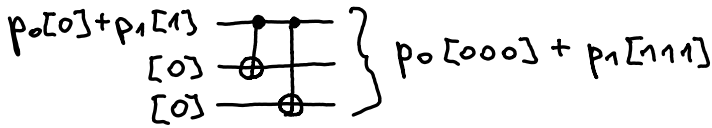
This list seems pretty daunting. However, neither of these obstacles are unique to quantum computing — they are all a consequence of linearity of the state space and operations, which is also the case for probabilistic bits. Let's try to tackle them first in this simpler setting and then generalize to the quantum case.

Error correction for probabilistic bits

We cannot copy probabilistic bits, but we can still use the 3-bit repetition code by linearity:

$$\text{Enc}: p_0[0] + p_1[1] \mapsto p_0[000] + p_1[111]$$

We can achieve this by using two CNOTs:



That's great, but how do we detect errors? Say, an X error happened on the first bit:

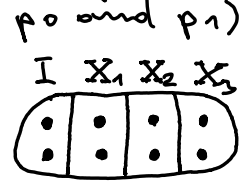
$$(X \otimes I \otimes I) \cdot (p_0[000] + p_1[111]) = p_0[100] + p_1[011].$$

We can't just measure the bits to compare them since this would collapse the distribution to one of the two cases and eliminate the appearance of p_0 and p_1. However, our goal is not to learn the values of individual bits, but rather which of the possible errors did occur. There are 4 cases:

$$\begin{aligned} \text{no error}: &\quad p_0[000] + p_1[111] \\ X_1: &\quad p_0[100] + p_1[011] \\ X_2: &\quad p_0[010] + p_1[101] \\ X_3: &\quad p_0[001] + p_1[110] \end{aligned}$$

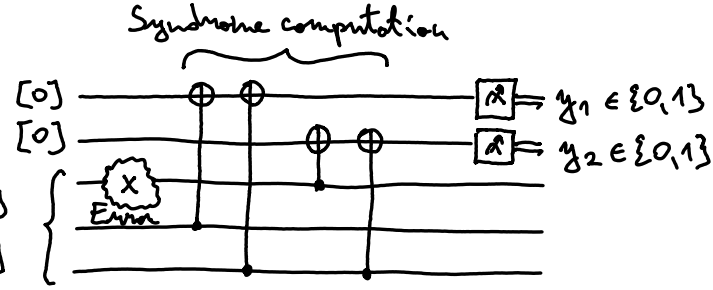
We see that all eight 3-bit strings occur, each exactly once. In fact, the errors partition these strings into 4 groups, each containing 2 strings (a string of Hamming weight 0 or 1, and its negation). All we want is to figure out in which of the four cases we are, without identifying a single individual string (as this would destroy p_0 and p_1):

	Error syndrome		
	y_1	y_2	
$x_1x_2x_3$	$x_2 \oplus x_3$	$x_1 \oplus x_3$	Error
000 111	0	0	I
100 011	0	1	X_1
010 101	1	0	X_2
001 110	1	1	X_3

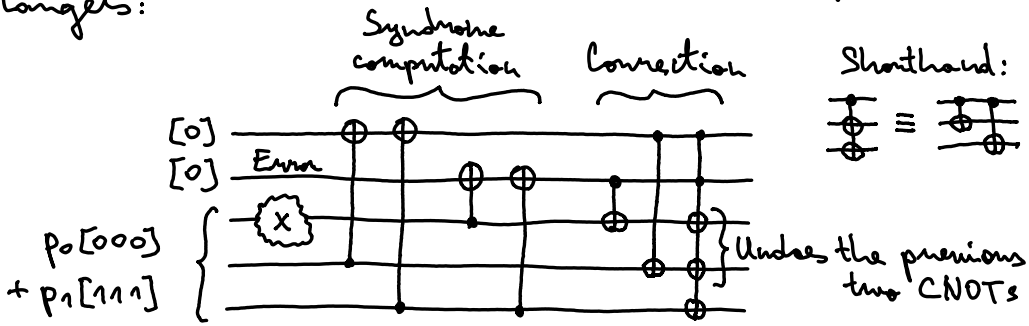


We can do this by inspecting the correlations between bits, for example the parities $y_1 = x_2 \oplus x_3$ and $y_2 = x_1 \oplus x_3$ uniquely identify which error occured. We can then correct it by applying X_y where $y = y_1 y_2$ is the index in binary.

This can be implemented as follows:



Note that we can start correcting the error on the fly before even measuring – we simply use the syndrome bits as controls and the data qubits as targets:



The syndrome bits can now be safely discarded and the data bits will be automatically corrected. This is somewhat similar to how a refrigerator keeps your food cool by pumping out the heat. You can actually keep track of how an X error propagates through the circuit by using the following identities:



This way you can move the errors to the right until they fall out the other end.

What about the final issue, namely dealing with continuous errors? For example, imagine the case when an X error is applied with some arbitrary probability q. This corresponds to the following operation:

$$Q = (1-q)\,I + q\,X = \begin{pmatrix} 1-q & q \\ q & 1-q \end{pmatrix}.$$

Assume this error happens on the first bit. The resulting state is

$$\begin{aligned} &\Big(\big((1-q)\,I + q\,X\big)\otimes I\otimes I\Big)\cdot\big(p_0\,[000] + p_1\,[111]\big) \\ &= (1-q)\cdot\big(p_0\,[000] + p_1\,[111]\big) \\ &\quad + q\cdot\big(p_0\,[100] + p_1\,[011]\big) \end{aligned}$$

By linearity, computing the syndrome results in two additional bits that take different values in the two cases (depending on whether I or X was applied on the first bit):

$$\begin{aligned} &(1-q)\cdot\,[00]\otimes\big(p_0\,[000] + p_1\,[111]\big) \\ &\quad + q\cdot\,[01]\otimes\big(p_0\,[100] + p_1\,[011]\big) \end{aligned}$$

Measuring the syndrome bits forces the error to become one of the two types, rather than a linear combination:

$$[00]\otimes\big(p_0\,[000] + p_1\,[111]\big)$$

$$\text{or}$$

$$[01]\otimes\big(p_0\,[100] + p_1\,[011]\big)$$

and we know which one it is from the measurement outcome, so we can easily correct it.

Assume we encode 1 bit into n bits to protect it against single-bit flip errors. How big does n have to be for this to work (e.g., could it be 2), and does it get easier or harder to protect the information against errors if we use more bits?

We can do a simple counting argument. The total dimension of an n bit code is 2^n. In this space we need to fit in $n+1$ 2-dimensional subspaces: there are n different possible bit-flip errors, plus the lucky case of no error, so $n+1$ cases we need to be able to discriminate. Hence, it must be that

$$2^n \geq 2(n+1)$$

The smallest n for which this holds is $n=3$, and as n increases, it gets easier and easier to fit $n+1$ 2-dimensional subspaces in a 2^n-dimensional space.

Quantum error correction

We can use a procedure that is identical to the 3-bit repetition code for probabilistic bits also in the quantum case. However, this protects only against X errors (or more generally, linear combinations of X and I). How about the most general single-qubit unitary error $U \in U(2)$? We can write

$$U = e^{i\varphi}\begin{pmatrix} a & b \\ -\bar{b} & \bar{a} \end{pmatrix}, \qquad |a|^2+|b|^2=1,$$

for some $\varphi \in \mathbb{R}$ and $a, b \in \mathbb{C}$. Let $a = t+iz$ and $b = y+ix$, for some $x, y, z, t \in \mathbb{R}$. Ignoring the global phase $e^{i\varphi}$,

$$U \sim \begin{pmatrix} t+iz & y+ix \\ -y+ix & t-iz \end{pmatrix} = tI + ixX + iyY + izZ,$$

$$t^2+x^2+y^2+z^2=1.$$

This is just the expansion of U as a linear combination of the four Pauli matrices (any complex 2×2 matrix has a unique such expansion since the Pauli matrices form an orthogonal matrix basis).

Assume this U error occurs on the first qubit of some quantum state $|\Psi\rangle$. The resulting state is

$$U_1|\Psi\rangle = t|\Psi\rangle + ix X_1|\Psi\rangle + iy Y_1|\Psi\rangle + iz Z_1|\Psi\rangle,$$

where the subscript denotes the qubit on which the gate acts. E.g., if $|\Psi\rangle$ has 3 qubits then $U_1 = U \otimes I \otimes I$.

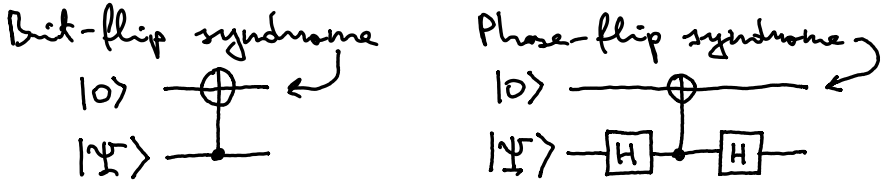
Since $iY = ZX$, we can think of the four terms as corresponding to four types of errors:

$$\begin{aligned} I &= \text{no error} \\ X &= \text{bit-flip} \\ Z &= \text{phase-flip} \\ ZX &= \text{bit- and phase-flip} \end{aligned}$$

Hence, if we are able to distinguish just these 4 cases, we can by linearity deal with any single-qubit unitary error — we simply compute the syndrome in a separate register, and by measuring this register force the continuous error U to collapse to one of these four cases.

To fix a general error U, it suffices to deal with just bit-flip and phase-flip errors, since the Y error can be dealt with by consecutively fixing both types of errors.

We can deal with bit-flip errors the same way as for probabilistic bits – by using a sequence of CNOTs to compute the parity of the desired qubits. For phase-flip errors, we can use the same circuit but conjugate the control qubits by H:



Here is another way to think about this. Since

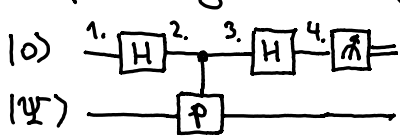


we can equivalently write the above two circuits as follows:



Note that this is phase estimation of Z and X up to one bit of precision. Recall that $Z^2 = X^2 = I$, so their eigenvalues are ± 1, and $1 = e^{2\pi i \cdot 0.0}$ while $-1 = e^{2\pi i \cdot 0.1}$ where the phase is written in binary. Also, the same as measuring X and Z observables.

If we measure the syndrome qubit, we project the data qubit to one of the eigenspaces of the corresponding Pauli operator P:



1. $|0\rangle|\Psi\rangle$ 2. $|+\rangle|\Psi\rangle = \frac{1}{\sqrt{2}}(|0\rangle+|1\rangle)|\Psi\rangle$

3. $\frac{1}{\sqrt{2}}(|0\rangle|\Psi\rangle + |1\rangle P|\Psi\rangle)$

4. $\frac{1}{2}((|0\rangle+|1\rangle)|\Psi\rangle + (|0\rangle-|1\rangle)P|\Psi\rangle)$

$= |0\rangle\otimes\left(\frac{I+P}{2}|\Psi\rangle\right) + |1\rangle\otimes\left(\frac{I-P}{2}|\Psi\rangle\right)$

Since P has eigenvalues ± 1, the operators $\frac{I \pm P}{2}$ have eigenvalues 0 and 1, and they are the projectors onto the ± 1 eigenspaces of P. Indeed, if $P = |\psi_0\rangle\langle\psi_0| - |\psi_1\rangle\langle\psi_1|$ then $I = |\psi_0\rangle\langle\psi_0| + |\psi_1\rangle\langle\psi_1|$ and $(I+P)/2 = |\psi_0\rangle\langle\psi_0|$ and $(I-P)/2 = |\psi_1\rangle\langle\psi_1|$.

This procedure is also known as <u>measuring the observable P</u>. For us, an <u>observable</u> is simply a Hermitian matrix ($P^\dagger = P$) such that $P^2 = I$. The above procedure projects a given state $|\psi\rangle$ onto either the $+1$ or -1 eigenspace of P, and outputs which of the two projections happened (this is the same as measuring in the eigenbasis of P). Note that for detecting X errors we measure the Z observable, while for detecting Z errors we measure X.

For example, to determine the parity of two qubits, we measure $Z \otimes Z$:

$$Z \otimes Z = \begin{pmatrix} 1 & & & \\ & -1 & & \\ & & -1 & \\ & & & 1 \end{pmatrix} \begin{matrix} 00 \\ 01 \\ 10 \\ 11 \end{matrix} \begin{matrix} \longrightarrow (+1)\text{-eigenspace} = \text{even parity} \\ \longrightarrow (-1)\text{-eigenspace} = \text{odd parity} \end{matrix}$$

Hence, the parity checks for the quantum version of the 3-bit code correspond to measuring the observables $I \otimes Z \otimes Z$ and $Z \otimes I \otimes Z$. The common $+1$ eigenspace of these is spanned by $|000\rangle$ and $|111\rangle$.

More generally, if two observables P and Q commute, i.e., $PQ = QP$, the order in which they are measured does not affect the probabilities of the outcomes. We say that such observables can be measured <u>simultaneously</u>. Indeed, commuting observables can be simultaneously diagonalized. This common basis can be used to unambiguously assign measurement outcomes to

each eigenvector. Intuitively, we can think of measuring an observable P as asking the question of whether the system has "property P". For commuting observables, the process of determining P does not affect the value of Q, so one can simultaneously ask to determine P and Q.

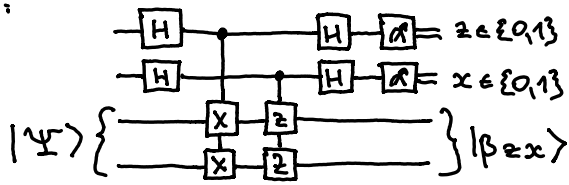
Example Let $P = X \otimes X$ and $Q = Z \otimes Z$. Note that $PQ = QP$. Bell states have the following eigenvalues for P and Q:

Bell states	$X \otimes X$	$Z \otimes Z$
$\lvert\beta_{00}\rangle = \frac{1}{\sqrt{2}}(\lvert 00\rangle + \lvert 11\rangle)$	+	+
$\lvert\beta_{01}\rangle = \frac{1}{\sqrt{2}}(\lvert 01\rangle + \lvert 10\rangle)$	+	−
$\lvert\beta_{10}\rangle = \frac{1}{\sqrt{2}}(\lvert 00\rangle - \lvert 11\rangle)$	−	+
$\lvert\beta_{11}\rangle = \frac{1}{\sqrt{2}}(\lvert 01\rangle - \lvert 10\rangle)$	−	−

In short:

$$(X \otimes X)\lvert\beta_{zx}\rangle = (-1)^z \lvert\beta_{zx}\rangle$$
$$(Z \otimes Z)\lvert\beta_{zx}\rangle = (-1)^x \lvert\beta_{zx}\rangle$$

The circuit for measuring these observables looks as follows:



This amounts to measuring $\lvert\Psi\rangle$ in the Bell basis — if the outcome is $z, x \in \{0,1\}$, the post-measurement state is $\lvert\beta_{zx}\rangle$. If $\lvert\Psi\rangle = (U \otimes I)\lvert\beta_{00}\rangle$, for some single-qubit errors $U = tI + ixX + iyY + izZ$ then, thanks to "local conversion",

$$\lvert\Psi\rangle = t\lvert\beta_{00}\rangle + ix\lvert\beta_{01}\rangle + iy\lvert\beta_{11}\rangle + iz\lvert\beta_{10}\rangle$$

is a linear combination of Bell states ($t^2 + x^2 + y^2 + z^2 = 1$). The above circuit measures this state in the Bell basis, producing $\lvert\beta_{zx}\rangle$ if the outcome is $z, x \in \{0,1\}$. We can restore this back to $\lvert\beta_{00}\rangle$ by, say, applying $Z^z X^x$ on the first qubit.

The above example lets us remove errors from the $|\beta_{00}\rangle$ state. However, it is not a quantum error correcting code, since it does not encode an arbitrary qubit state – it merely helps when dealing with this particular state. Is a 2- or 3-qubit code even possible?

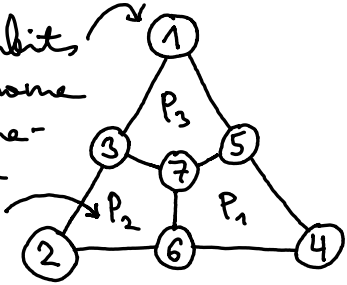
Let us do some counting to see how many qubits we need to protect a single qubit against Pauli errors. If we encode the original single-qubit state into n qubits, the total dimension is 2^n. There are n qubits and on each of them can occur one of three possible Pauli errors, plus the lucky case when no error occurs at all. Hence, we need to distinguish between $3n+1$ different cases, which means we need to be able to fit $3n+1$ mutually orthogonal 2-dimensional subspaces into 2^n dimensions:

$$2^n \geq 2(3n+1).$$

This is possible only when $n \geq 5$, and indeed there is a 5 qubit code. However, there is also a somewhat simpler 7 qubit code.

Steane code

It involves 7 qubits and the syndrome is obtained by measuring six observables P_1, P_2, P_3, Q_1, Q_2, Q_3.



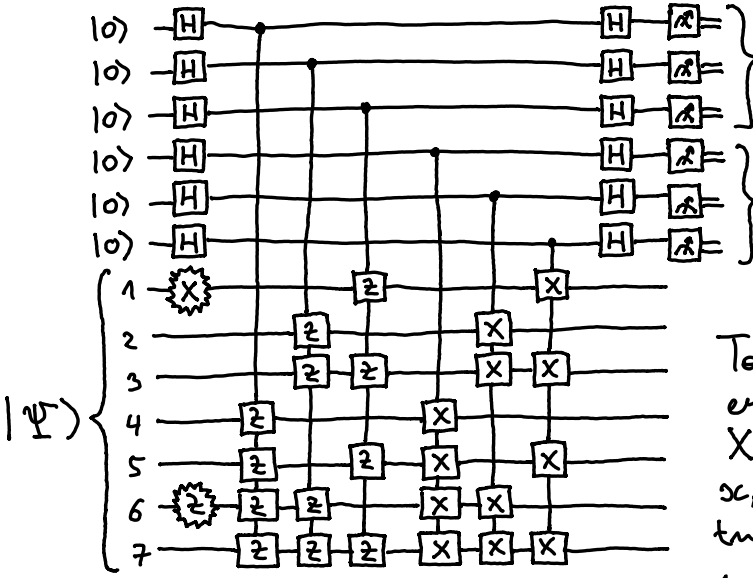
	1	2	3	4	5	6	7
$P_1 =$				Z	Z	Z	Z
$P_2 =$		Z	Z			Z	Z
$P_3 =$	Z		Z		Z		Z

$Q_1 =$				X	X	X	X
$Q_2 =$		X	X			X	X
$Q_3 =$	X		X		X		X

They mutually pairwise commute since any two of the plaquettes have an even number of vertices in common (the corners of the plaquette are the qubits on which the observable acts non-trivially).

There are 6 observables and hence 6 syndrome qubits, so $2^6 = 64$ different possible syndromes. Recall that there are $3n+1 = 22$ different discrete single-qubit errors on $n=7$ qubits. In fact, Steane code can correct not only those, but also the more general errors that correspond to X on at most one qubit and Z on at most one (possibly different) qubit. There are, exactly $(7+1)\cdot(7+1) = 8\cdot 8 = 64$ such errors, perfectly matching the number of syndromes. It turns out each of these 64 errors moves the state to a different 2-dimensional subspace, and these subspaces are mutually orthogonal, thus totalling to $2\cdot 64 = 128 = 2^7$ dimensions, which fit exactly in 7 qubits.

The full syndrome measurement circuit is as follows:



To correct the errors, we apply X_x and Z_z, where $x, z \in \{1, \dots, 7\}$ are the two halves of the syndrome. If $x=0$ or $z=0$, we do not need to apply that correction. Note that, just like in the example with the Bell states, Z observables detect X errors and the other way round.

This constitutes the Fix operation from page 2.

What remains to discuss are the states that represent $|0\rangle$ and $|1\rangle$, and the circuit for encoding a general qubit state $|\psi\rangle \in \mathbb{C}^2$. Those states have no error, so they must be $(+1)$-eigenvectors of all 6 observables simultaneously.

Since $Z|a\rangle = (-1)^a |a\rangle$ for all $a \in \{0,1\}$, a computational basis state $|a_1 \dots a_n\rangle$ is a $(+1)$-eigenvector of $Z_1^{b_1} \otimes \dots \otimes Z_n^{b_n}$ if $a_1 b_1 \oplus a_2 b_2 \oplus \dots \oplus a_n b_n = a \cdot b = 0$. Since each Z observable has an even number of Z's, and any two plaquets overlap in an even number of positions, the three bitstrings

$$\begin{array}{c} 0001111 \\ 0110011 \\ 1010101 \end{array}$$

representing the three plaquets already have this property, and so does any linear combination (over $\mathbb{Z}_2$). There are $2^3 = 8$ such combinations:

$$\begin{array}{c} 0000000 \\ 1010101 \\ 0110011 \\ 1100110 \\ 0001111 \\ 1011010 \\ 0111100 \\ 1101001 \end{array}$$

The corresponding computational basis states span an 8-dimensional subspace of $(\mathbb{C}^2)^{\otimes 7}$ that is invariant under all three Z observables. The X observables permute these eight states, so their uniform superposition is left invariant also by all three X observables. We use this superposition to encode the $|0\rangle$ state. The $|1\rangle$ is encoded by the same superposition, but with X applied to all 7 qubits.

More explicitly:

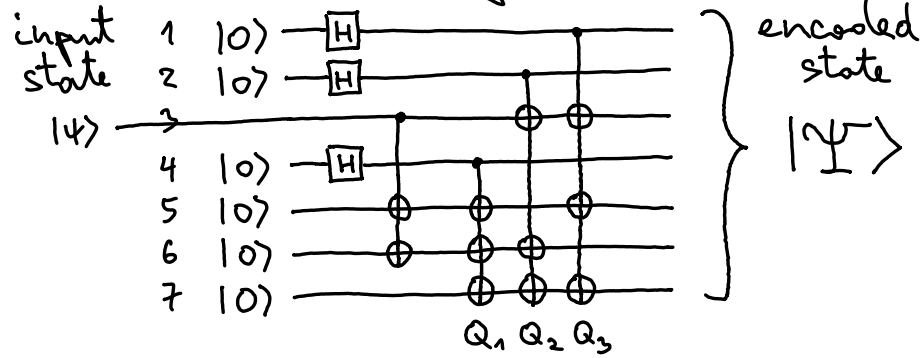
$$|0\rangle \mapsto \frac{1}{\sqrt{8}}\big(\,|00000000\rangle + |10101010\rangle + |01100110\rangle + |11001100\rangle + |00011110\rangle + |10110100\rangle + |01111000\rangle + |11010001\rangle\big)$$

$$|1\rangle \mapsto \frac{1}{\sqrt{8}}\big(\,|11111111\rangle + |01010101\rangle + |10011001\rangle + |00110011\rangle + |11100001\rangle + |01001011\rangle + |10000111\rangle + |00101101\rangle\big)$$

Or, for $x \in \{0,1\}$,

$$|x\rangle \mapsto \frac{1}{\sqrt{8}} \sum_{b_0,b_1,b_2 \in \{0,1\}} |x\oplus b_2, x\oplus b_1, x\oplus b_1\oplus b_2, x\oplus b_0, x\oplus b_0\oplus b_2, x\oplus b_0\oplus b_1, x\oplus b_0\oplus b_1\oplus b_2\rangle$$

To produce a superposition of these two states with the right coefficients from $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, one can use the following circuit:



It is based on the observation that qubits 1, 2, 4 take each of the $2^3 = 8$ values in exactly one of the terms. This superposition is created by applying H on these three qubits. The last three gates produce the correct values on the remaining four qubits by effectively applying the three X observables Q_i. The remaining gate guarantees that all bits get inverted when $|\psi\rangle = |1\rangle$. This works because

$$Q_1 Q_2 Q_3 \, X^{\otimes 7} = \left(X^{\otimes 6} \otimes I\right) Q_1 Q_2 Q_3 ,$$

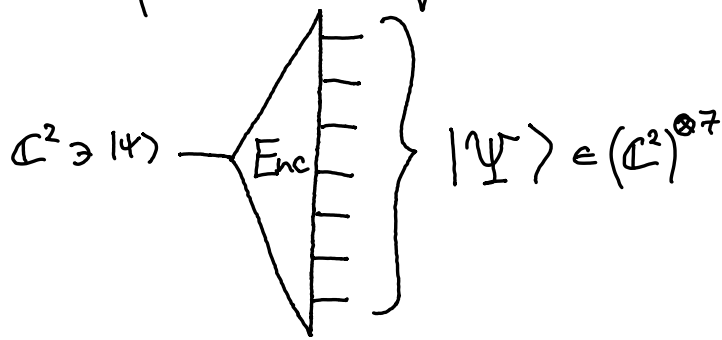
so we only need to negate the first six qubits if we want to encode $|1\rangle$.

Since qubits 1, 2, 4 are prepared in state

$$H|0\rangle = |+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$$

and $X|+\rangle = (|1\rangle + |0\rangle)/\sqrt{2} = |+\rangle$, we can skip the X gates on them. So if qubit 3 is $|1\rangle$, we only need to negate qubits 5 and 6. This explains the additional gate in the circuit.

This constitutes a description of the encoding circuit Enc from 1 to 7 qubits



The decoding circuit Dec is identical to the Enc circuit run backwards.

Fault tolerance

The scheme described above protects only against errors that occur inbetween two Fix operations. However, an error can occur at any time, even during a Fix operation. Dealing with this requires more sophisticated tricks that are part of the theory of fault tolerance.